Experiment No: 09

Title: Study & Implementation of SQL Triggers

Topics to be covered:

- Introduction to Triggers
- Types of Triggers
- Creating and Dropping Triggers
- Examples of Triggers

Introduction to Triggers

A SQL Trigger is a stored program that is automatically executed or fired when certain events occur in a database table. Triggers can be used for maintaining data integrity, enforcing business rules, auditing, and more.

*In **DBMS (Database Management Systems)**, triggers are special procedures that are automatically executed or fired when specific events occur in a database. Triggers are usually used for maintaining the integrity of the information on the database.*

*A **trigger** is a database object that automatically executes or fires when certain events occur. Triggers help enforce business rules, data integrity, and automate system tasks.*

Key Points:

- Trigger is invoked automatically.
- Can be set on INSERT, UPDATE, DELETE events.
- Useful for maintaining consistent data.

Types of Triggers in SQL

1. BEFORE INSERT Trigger
2. AFTER INSERT Trigger
3. BEFORE UPDATE Trigger
4. AFTER UPDATE Trigger
5. BEFORE DELETE Trigger
6. AFTER DELETE Trigger

Procedure

1. Create sample tables (e.g., employees and employee_audit).
2. Create triggers for different operations (INSERT, UPDATE, DELETE).
3. Perform actions on the table to invoke triggers.
4. Observe the effects and validate trigger behavior.

SQL Trigger Examples

Table Setup

Employees table:

```
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    name VARCHAR(50),
    salary DECIMAL(10, 2)
);
```

Audit table to store logs:

```
CREATE TABLE employee_audit (
    audit_id INT AUTO_INCREMENT PRIMARY KEY,
    emp_id INT,
    action_type VARCHAR(20),
    old_salary DECIMAL(10,2),
    new_salary DECIMAL(10,2),
    action_time DATETIME
);
```

Creating and Dropping Triggers

Syntax for Creating a Trigger:

```
CREATE TRIGGER trigger_name
{BEFORE | AFTER} {INSERT | UPDATE |
DELETE}
ON table_name
FOR EACH ROW
BEGIN
    -- trigger body
END;
```

```
CREATE TRIGGER before_insert_employees
BEFORE INSERT ON employees
FOR EACH ROW
BEGIN
IF NEW.salary < 10000 THEN
        SET NEW.salary = 10000;
    END IF;

END;
```

Syntax for Dropping a Trigger:

```
DROP TRIGGER trigger_name;
```

```
DROP TRIGGER before_insert_employees;
```

BEFORE INSERT Trigger

This trigger runs **before a new record** is added.

```
DELIMITER $$

CREATE TRIGGER before_insert_employees
BEFORE INSERT ON employees
FOR EACH ROW
BEGIN
    IF NEW.salary < 10000 THEN
        SET NEW.salary = 10000;
    END IF;
END$$

DELIMITER ;
```

Explanation:

If someone tries to insert an employee with salary 8000, it will automatically be changed to 10000.

Test Data:

```
INSERT INTO employees VALUES (1, 'Ridwan', 8000);
SELECT * FROM employees;
```

Output:

```
+--------+--------+---------+
| emp_id | name   | salary  |
+--------+--------+---------+
| 1      | Ridwan | 10000.00|
+--------+--------+---------+
```

AFTER INSERT Trigger

This trigger runs **after a new record** is added.

```
DELIMITER $$

CREATE TRIGGER after_insert_employees
AFTER INSERT ON employees
FOR EACH ROW
BEGIN
    INSERT INTO employee_audit(emp_id, action_type, old_salary,
new_salary, action_time)
    VALUES (NEW.emp_id, 'INSERT', NULL, NEW.salary, NOW());
END$$
```

```
DELIMITER ;
```

Explanation:
Whenever a new employee is added, this automatically logs the event in the employee_audit table.

Test Data:

```
INSERT INTO employees VALUES (2, 'Mohammad Rafiq', 12000);
SELECT * FROM employees;
SELECT * FROM employee_audit;
```

Output:

```
+--------+----------------+---------+
| emp_id | name           | salary  |
+--------+----------------+---------+
| 1      | Ridwan         | 10000.00|
| 2      | Mohammad Rafiq | 12000.00|
+--------+----------------+---------+

+----------+--------+-------------+------------+-------------+---------------------+
| audit_id | emp_id | action_type | old_salary | new_salary  | action_time         |
+----------+--------+-------------+------------+-------------+---------------------+
| 1        | 2      | INSERT      | NULL       | 12000.00    | 2025-10-27 13:00:00 |
+----------+--------+-------------+------------+-------------+---------------------+
```

BEFORE UPDATE Trigger
This trigger runs **before a record is updated**.

```
DELIMITER $$

CREATE TRIGGER before_update_employees
BEFORE UPDATE ON employees
FOR EACH ROW
BEGIN
    IF NEW.salary < OLD.salary THEN
        SET NEW.salary = OLD.salary;
    END IF;
END$$

DELIMITER ;
```

Explanation:
If someone tries to lower an employee's salary, the change won't be allowed — it keeps the old salary.\

Test Data:

```
UPDATE employees SET salary = 9000 WHERE emp_id = 2;
SELECT * FROM employees;
```

Output:

```
+--------+----------------+---------+
| emp_id | name           | salary  |
+--------+----------------+---------+
| 1      | Ridwan         | 10000.00|
| 2      | Mohammad Rafiq | 12000.00|
+--------+----------------+---------+
```

(Salary not changed since 9000 < 12000.)

AFTER UPDATE Trigger

This trigger runs **after a record is updated**.

```
DELIMITER $$

CREATE TRIGGER after_update_employees
AFTER UPDATE ON employees
FOR EACH ROW
BEGIN
    INSERT INTO employee_audit(emp_id, action_type, old_salary,
new_salary, action_time)
    VALUES (NEW.emp_id, 'UPDATE', OLD.salary, NEW.salary, NOW());
END$$

DELIMITER ;
```

Explanation:

Whenever an employee's salary is changed, this trigger stores both the previous and updated values in the audit table.

Test Data:

```
UPDATE employees SET salary = 13000 WHERE emp_id = 1;
SELECT * FROM employee_audit;
```

Output:

```
+----------+--------+-------------+------------+-------------+---------------------+
| audit_id | emp_id | action_type | old_salary | new_salary  | action_time         |
+----------+--------+-------------+------------+-------------+---------------------+
| 1        | 2      | INSERT      | NULL       | 12000.00    | 2025-10-27 13:00:00 |
| 2        | 1      | UPDATE      | 10000.00   | 13000.00    | 2025-10-27 13:02:00 |
+----------+--------+-------------+------------+-------------+---------------------+
```

BEFORE DELETE Trigger

This trigger runs **before a record is deleted**.

```
DELIMITER $$

CREATE TRIGGER before_delete_employees
BEFORE DELETE ON employees
FOR EACH ROW
BEGIN
    IF OLD.name = 'Ridwan' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Cannot delete Admin user';
    END IF;
END$$

DELIMITER ;
```

Explanation:

If anyone tries to delete the 'Ridwan' employee, an error message is shown and the action is blocked.

Test Data:

```
DELETE FROM employees WHERE name = 'Ridwan';
```

Output:

```
ERROR 1644 (45000): Cannot delete Ridwan record
```

AFTER DELETE Trigger

This trigger runs **after a record is deleted**.

```
DELIMITER $$

CREATE TRIGGER after_delete_employees
AFTER DELETE ON employees
FOR EACH ROW
BEGIN
    INSERT INTO employee_audit(emp_id, action_type, old_salary,
new_salary, action_time)
    VALUES (OLD.emp_id, 'DELETE', OLD.salary, NULL, NOW());
END$$

DELIMITER ;
```

Explanation:

Every time an employee is deleted, their info is saved in the audit table for future reference.

Test Data:

```
DELETE FROM employees WHERE name = 'Abdur Rahim';
SELECT * FROM employee_audit;
```

Output:

```
+----------+--------+-------------+------------+-------------+---------------------+
| audit_id | emp_id | action_type | old_salary | new_salary  | action_time         |
+----------+--------+-------------+------------+-------------+---------------------+
| 1        | 2      | INSERT      | NULL       | 12000.00    | 2025-10-27 13:00:00 |
| 2        | 1      | UPDATE      | 10000.00   | 13000.00    | 2025-10-27 13:02:00 |
| 3        | 3      | DELETE      | 15000.00   | NULL        | 2025-10-27 13:04:00 |
+----------+--------+-------------+------------+-------------+---------------------+
```

Final Table States

Employees Table:

```
SELECT * FROM employees;
+--------+----------------+---------+
| emp_id | name           | salary  |
+--------+----------------+---------+
| 1      | Ridwan         | 13000.00|
| 2      | Mohammad Rafiq | 12000.00|
+--------+----------------+---------+
```

Employee Audit Table:

```
SELECT * FROM employee_audit;
+----------+--------+-------------+------------+-------------+---------------------+
| audit_id | emp_id | action_type | old_salary | new_salary  | action_time         |
+----------+--------+-------------+------------+-------------+---------------------+
| 1        | 2      | INSERT      | NULL       | 12000.00    | 2025-10-27 13:00:00 |
| 2        | 1      | UPDATE      | 10000.00   | 13000.00    | 2025-10-27 13:02:00 |
| 3        | 3      | DELETE      | 15000.00   | NULL        | 2025-10-27 13:04:00 |
+----------+--------+-------------+------------+-------------+---------------------+
```

Lab Tasks-1:

1. Create a trigger to automatically insert a record into an audit table when a new employee is added.
2. Create a trigger to update a field `last_modified` every time an employee's data is updated.
3. Prevent deletion of employees with salary greater than 80000.

Lab Tasks-2:

1. Create a trigger to log each employee INSERT action into an audit table.
2. Create a trigger to update a last_modified timestamp whenever data is updated.
3. Prevent deletion of employees with salary greater than 80000.
4. Automatically increase salary by 10% if an employee is promoted.
5. Create a trigger to count total number of employees after each insert.
6. Prevent inserting employees with NULL names.
7. Create a trigger that logs username and time of every DELETE operation.
8. Prevent updates to employee_id field.
9. Copy deleted employee data into a backup table automatically.
10. Alert if salary increment exceeds 50% of old salary

Lab Task-3:

Create the following tables first:

```
CREATE TABLE departments (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50),
    total_employees INT DEFAULT 0
);

CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    name VARCHAR(50),
    salary DECIMAL(10,2),
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);

CREATE TABLE employee_audit (
    audit_id INT AUTO_INCREMENT PRIMARY KEY,
    emp_id INT,
    action_type VARCHAR(20),
    old_salary DECIMAL(10,2),
    new_salary DECIMAL(10,2),
    action_time DATETIME
);
```

Questions

1. **Trigger for Automatic Employee Counting**
   Create an **AFTER INSERT trigger** on the `employees` table to **increase** the corresponding department's `total_employees` count by 1 when a new employee joins.

2. **Trigger for Employee Exit**
   Create an **AFTER DELETE trigger** on the `employees` table to **decrease** the `total_employees` count in the `departments` table when an employee leaves.

3. **Salary Validation Trigger**
   Create a **BEFORE INSERT trigger** that prevents inserting an employee with a salary less than **10,000**. If it is less, automatically set it to **10,000**.

4. **Salary Update Audit Trigger**
   Create an **AFTER UPDATE trigger** that logs old and new salary values into `employee_audit` every time an employee's salary is updated.

5. **Department Transfer Trigger**
   Create an **AFTER UPDATE trigger** to **update the `total_employees` counts** when an employee changes departments (i.e., the `dept_id` is updated).

6. **High Salary Protection Trigger**
   Create a **BEFORE UPDATE trigger** that **prevents lowering** an employee's salary if the new salary is less than the old salary.

7. **Delete Protection Trigger**
   Create a **BEFORE DELETE trigger** that prevents deletion of employees who belong to the department named `'Management'`.

8. **Salary Bonus Logging**
   Create an **AFTER UPDATE trigger** to insert a record in `employee_audit` whenever an employee's salary increases by more than 20%.

9. **Project Allocation Trigger (New Table)**
   Create a new table `project_assignments(emp_id INT, project_name VARCHAR(50)).`
   Then, create a **BEFORE INSERT trigger** that prevents assigning a project to an employee **who does not exist** in the `employees` table.

10. **Comprehensive Delete Audit**
    Create an **AFTER DELETE trigger** that stores the deleted employee's ID, name, and last salary into `employee_audit` with action_type `'DELETE'`.